TECHNOVA SOLUTIONS PVT. LTD.

CODING STANDARDS

Document ID	TN-ENG-STD-002
Document Type	Engineering Coding Standards
Document Owner	Engineering
Approving Authority	VP Engineering / CTO
Classification	Internal — Engineering
Version	1.0
Effective Date	01 April 2026
Review Cycle	Annual
Status	Active
Supersedes	New Standard

1. Purpose

These Coding Standards define the minimum engineering practices expected when writing, modifying, reviewing, and maintaining software at TechNova Solutions Pvt. Ltd. (“TechNova” or “the Company”).

The standards are intended to improve readability, consistency, maintainability, security, testability, and long-term ownership of the Company's codebase.

2. Scope

These standards apply to production software, internal applications, services, scripts, automation, libraries, data-processing code, infrastructure code, and other maintained engineering code unless a documented project-specific standard provides a more appropriate requirement.

Language-specific conventions may be maintained in project repositories while remaining consistent with the principles in this document.

3. General Coding Principles

- Write code for clarity before cleverness.
- Prefer simple solutions that satisfy the actual requirement.
- Keep modules and functions focused on a clear responsibility.
- Make behavior predictable and explicit.
- Minimize unnecessary coupling.
- Prefer maintainable abstractions over premature abstraction.
- Treat security, reliability, and observability as part of implementation quality.
- Leave code and documentation in a condition that another engineer can understand.

4. Naming Conventions

Names should communicate intent and should be understandable without requiring extensive comments.

- Use descriptive names for variables, functions, classes, modules, services, and configuration values.

- Avoid unexplained abbreviations.
- Use established terminology consistently across the project.
- Boolean names should make true/false meaning clear.
- Constants should follow the language or project convention and clearly represent immutable values.
- Do not use names that obscure a variable's purpose merely to make code shorter.

5. Functions and Methods

- Keep functions focused on one logical responsibility.
- Prefer small, readable functions over large blocks of mixed responsibilities.
- Use parameters that clearly represent required inputs.
- Avoid excessive parameter lists; use an appropriate data structure when a group of related values belongs together.
- Keep side effects explicit where possible.
- Return values should have clear and predictable meaning.

6. Classes and Modules

- Classes should have a clear responsibility.
- Modules should group related functionality.
- Avoid classes that accumulate unrelated responsibilities.
- Prefer composition when it produces clearer ownership than inheritance.
- Expose only the interfaces required by callers.
- Keep implementation details private where the language supports appropriate visibility.

7. Code Structure

Code should be organized so that the normal execution path is easy to follow.

- Keep related logic close together.
- Separate business logic from infrastructure concerns where practical.
- Avoid deeply nested conditional structures.
- Use guard clauses where they improve readability.
- Keep configuration separate from hard-coded business logic.
- Remove dead or unreachable code.

8. Formatting

All repositories should use an agreed formatter or formatting convention appropriate to the language.

- Use consistent indentation.
- Use consistent line endings and file encoding.
- Keep line length within the project convention where practical.
- Do not mix formatting styles within a repository.
- Automated formatters should be preferred over manual formatting when available.

9. Comments

Comments should explain intent, reasoning, constraints, or non-obvious behavior rather than restating what the code already says.

- Document important design decisions.
- Explain non-obvious workarounds and constraints.
- Update comments when behavior changes.
- Avoid comments that merely repeat variable or function names.
- Remove obsolete comments.

10. Documentation Strings and API Documentation

Public or shared functions, classes, APIs, and important modules should have appropriate documentation.

Documentation should explain purpose, inputs, outputs, important side effects, exceptions or error conditions, and usage constraints where relevant.

11. Error Handling

- Handle errors at the layer that can make an appropriate decision.
- Do not silently ignore unexpected errors.
- Use meaningful error types or messages.
- Do not expose sensitive information through error messages.
- Preserve useful diagnostic context when propagating errors.
- Avoid using exceptions or equivalent mechanisms for ordinary control flow when clearer alternatives exist.

12. Input Validation

Inputs originating outside a trusted component boundary should be treated as untrusted.

- Validate required fields and expected formats.
- Validate ranges, sizes, and allowed values where applicable.
- Reject malformed input safely.
- Perform authorization checks independently from basic input validation.
- Do not rely solely on client-side validation.

13. Security Requirements

- Follow secure coding practices appropriate to the technology.
- Never hard-code passwords, API keys, tokens, private keys, or other secrets.
- Use parameterized queries or safe database APIs.
- Protect against injection, unsafe deserialization, path traversal, and other relevant input-driven risks.
- Apply least privilege to service and application access.
- Do not disable security controls merely to simplify development.

14. Secrets Management

Secrets must be stored and accessed using approved mechanisms.

- Do not commit secrets to source control.
- Do not place secrets in source comments or sample configuration committed to repositories.

- Use environment-specific secret management where appropriate.
- Rotate credentials when exposure is suspected.
- Remove accidentally committed secrets from active use immediately and follow incident procedures.

15. Dependency Management

- Use approved package managers and repositories.
- Prefer maintained and appropriately licensed dependencies.
- Keep dependencies reasonably current.
- Review security advisories affecting production dependencies.
- Remove unused dependencies.
- Pin or constrain versions where required for reproducibility and stability.

16. Third-Party Code

Third-party libraries and copied code must be evaluated for security, licensing, maintenance, and compatibility.

Engineers must comply with applicable open-source and third-party licensing requirements.

17. Database Coding

- Use parameterized queries or equivalent safe APIs.
- Handle transactions deliberately.
- Validate database operation failures.
- Avoid unnecessary queries and repeated database access in performance-sensitive paths.
- Use migrations or approved schema-management mechanisms for structural changes.
- Do not expose sensitive database details in application errors.

18. API Design

- Use consistent resource, operation, and field naming.
- Validate request inputs.
- Authenticate and authorize protected operations.
- Return appropriate status or error information.
- Do not expose internal implementation details unnecessarily.
- Version externally consumed APIs when breaking changes cannot be avoided.
- Document important contracts and compatibility expectations.

19. Logging

Application logs should support troubleshooting and operational visibility without unnecessarily exposing sensitive information.

- Log significant failures and security-relevant events.
- Include enough context to investigate an issue.
- Do not log passwords, authentication tokens, private keys, or unnecessary personal data.
- Use appropriate log levels.

- Avoid excessive logging that creates operational noise or unnecessary storage cost.

20. Observability

Services should provide appropriate metrics, logs, traces, health indicators, or other observability mechanisms based on their operational importance.

Engineers should consider observability during implementation rather than treating it solely as a post-release activity.

21. Concurrency and Asynchronous Code

- Make shared-state access explicit and safe.
- Avoid unnecessary mutable shared state.
- Use established concurrency primitives and framework patterns.
- Handle cancellation and timeouts where appropriate.
- Consider race conditions and deadlocks in concurrent code.
- Document unusual concurrency assumptions.

22. Resource Management

- Close or release files, connections, streams, locks, and other resources appropriately.
- Use language-supported resource-management constructs where available.
- Avoid unbounded memory, thread, connection, or queue growth.
- Consider resource limits and failure behavior.

23. Performance

Performance optimization should be based on actual requirements, measurements, or credible risk rather than speculation.

- Prefer clear algorithms first.
- Consider algorithmic complexity for significant data-processing paths.
- Measure before and after meaningful optimizations.
- Avoid unnecessary network calls and repeated expensive operations.
- Document important performance assumptions.

24. Testing Requirements

New or changed behavior should have appropriate automated tests based on risk and complexity.

- Test normal behavior.
- Test important boundary conditions.
- Test expected error behavior.
- Include regression tests for significant defects.
- Keep tests deterministic where practical.
- Follow the separate Testing Standards document for detailed test requirements.

25. Testability

Code should be structured so important behavior can be tested without unnecessary dependence on external systems.

- Keep business logic separable from external dependencies where practical.
- Use dependency injection or equivalent patterns when beneficial.
- Avoid hidden global state.
- Make time, randomness, network calls, and other difficult dependencies controllable in tests where appropriate.

26. Code Duplication

Avoid unnecessary duplication of complex or frequently changing logic.

Duplication should not automatically be removed when abstraction would make the code harder to understand or maintain.

27. Abstraction

- Introduce abstractions when they clarify a stable concept.
- Avoid creating layers solely for theoretical future reuse.
- Keep interfaces small and purposeful.
- Prefer abstractions that reflect real domain or technical boundaries.

28. Configuration

Configuration should be explicit, validated, and separated from source code where appropriate.

- Do not hard-code environment-specific endpoints or credentials.
- Validate required configuration during startup or an appropriate initialization phase.
- Use safe defaults only where the default is genuinely appropriate.
- Document important configuration options.

29. Feature Flags

Feature flags may be used to control rollout, testing, or operational behavior.

- Give each flag a clear owner and purpose.
- Define expected lifecycle and removal criteria.
- Avoid accumulating permanent unused flags.
- Protect sensitive operational flags with appropriate access controls.

30. Backward Compatibility

Changes to shared libraries, APIs, schemas, and service contracts should consider existing consumers.

- Identify affected consumers before breaking changes.
- Prefer compatible evolution where practical.
- Provide migration guidance for necessary breaking changes.
- Remove deprecated interfaces according to an agreed lifecycle.

31. Data and Privacy

- Use only the data necessary for the intended purpose.
- Do not expose personal or confidential data unnecessarily.
- Use appropriate anonymization or synthetic data in development and testing when practical.

- Follow approved data-retention and access requirements.
- Treat production data as sensitive unless explicitly classified otherwise.

32. AI and Machine Learning Code

AI and machine-learning implementations must follow approved Engineering, Information Security, Access Control, and AI governance requirements.

- Track relevant model, library, and data dependencies.
- Validate inputs and outputs.
- Protect prompts, datasets, embeddings, model artifacts, and credentials.
- Do not expose restricted training or retrieval data.
- Evaluate important model behavior before production use.
- Maintain reproducibility information appropriate to the system.

33. Retrieval and Knowledge Systems

Code implementing document ingestion, chunking, indexing, retrieval, reranking, context construction, and AI response generation must preserve authorization boundaries.

- Apply document and user permissions before protected content enters downstream generation context.
- Do not rely on the final response layer alone to hide unauthorized information.
- Maintain traceability between retrieved chunks and source documents where required.
- Ensure access-control changes propagate to retrieval systems.

34. Infrastructure as Code

- Store infrastructure definitions in approved version-control repositories.
- Use repeatable and reviewable changes.
- Do not embed secrets in infrastructure definitions.
- Use environment-specific configuration safely.
- Review changes to production infrastructure.

35. Generated Code

Code generated by AI assistants, code generators, or other automated tools remains the responsibility of the engineer who integrates it.

- Review generated code for correctness.
- Check security implications.
- Check licensing or provenance concerns where relevant.
- Add appropriate tests.
- Do not assume generated code is correct merely because it compiles or passes a superficial check.

36. Dead Code and Cleanup

Unused functions, obsolete flags, unreachable branches, stale comments, and unnecessary dependencies should be removed when identified and when removal is safe.

37. TODO and FIXME Items

TODO, FIXME, and similar markers should be specific enough to explain the outstanding work.

Important technical or security risks should not be hidden indefinitely behind an informal TODO; they should be tracked through the appropriate work-management process.

38. Exception Handling in Production

Production code should fail safely and provide useful operational signals.

- Avoid exposing stack traces or internal details to end users.
- Use appropriate fallback behavior when dependencies fail.
- Use timeouts for operations that may otherwise block indefinitely.
- Do not retry operations blindly when retries can amplify failures.

39. Maintainability

- Prefer consistent patterns within a codebase.
- Refactor code when complexity materially affects correctness or delivery.
- Keep modules understandable for engineers other than the original author.
- Document unusual architectural or operational constraints.

40. Code Quality Gates

Projects may enforce automated quality gates such as formatting checks, linting, static analysis, dependency scanning, unit tests, security checks, or coverage thresholds.

Required gates must pass before code is integrated or released unless an authorized exception is documented.

41. Local Development

- Use supported language and tooling versions.
- Follow repository setup instructions.
- Do not use production credentials for local development unless explicitly authorized and protected.
- Keep local configuration and secrets outside source control.

42. Commit Quality

Commits should represent understandable engineering changes.

- Use concise, meaningful commit messages.
- Keep unrelated changes separate when practical.
- Do not commit generated artifacts unless the repository explicitly requires them.
- Do not commit secrets, credentials, or unnecessary local files.

43. Review Readiness

Before submitting code for review, the author should verify that the change is complete enough for meaningful review.

- Run applicable tests.
- Run required formatting and lint checks.
- Review the diff for accidental changes.
- Remove debugging statements and temporary code.

- Update documentation when behavior or interfaces change.

44. Refactoring

Refactoring should preserve intended behavior unless behavior change is explicitly part of the work.

Large refactors should be planned and reviewed according to their risk and impact.

45. Breaking Changes

Breaking changes should be explicitly identified and communicated to affected stakeholders.

Where practical, provide migration guidance, compatibility periods, or controlled rollout mechanisms.

46. Documentation of Technical Decisions

Significant decisions involving architecture, data models, dependencies, security, performance, or operational behavior should be documented using the team's approved mechanism.

47. Code Ownership

Teams should maintain clear ownership of important repositories, services, and components.

Ownership does not remove the responsibility of other engineers to write maintainable code and raise concerns.

48. Prohibited Coding Practices

- Hard-coding secrets.
- Disabling security controls without authorization.
- Introducing knowingly malicious code.
- Silently swallowing important errors.
- Copying restricted or proprietary code without authorization.
- Committing credentials or sensitive data.
- Falsifying tests, benchmarks, or quality results.
- Using production access for unauthorized experimentation.

49. Exceptions

A project may request an exception where a standard is technically inappropriate or creates a disproportionate constraint.

Exceptions should identify the affected requirement, justification, risk, compensating controls, owner, and duration, and require appropriate approval.

50. Frequently Asked Questions

Should every function be very small?

No. Functions should be focused and readable; an arbitrary line-count rule is less important than clear responsibility.

Can I use a different naming style?

Follow the established language and repository convention unless an approved project standard says otherwise.

Can I commit an API key temporarily?

No. Secrets must not be committed to source control, even temporarily.

Can AI-generated code be used?

Yes, where permitted, but the engineer remains responsible for correctness, security, testing, and applicable licensing or policy requirements.

Do all code changes need automated tests?

Changed behavior should receive appropriate testing based on risk and complexity; detailed requirements are defined by Testing Standards.

Can I optimize code before measuring it?

Simple, low-risk improvements are fine, but significant performance changes should be supported by requirements or measurement.

Can I copy code from the internet?

Only when its license, provenance, security, and use are acceptable for the project.

What should I do with a security concern in code?

Raise it promptly through the appropriate Engineering or Information Security process.

Can I leave TODO comments?

Yes, when they clearly describe legitimate follow-up work; important risks should also be tracked appropriately.

Who is responsible for generated code?

The engineer or team integrating it is responsible for reviewing and validating it.

51. Standard Administration

Engineering owns these Coding Standards and reviews them at least annually or when material changes occur in technology, security requirements, development practices, or organizational needs.

The latest approved version maintained in the official Company repository is the authoritative version.

52. Revision History

Version	Effective Date	Description	Owner	Approval
1.0	01 April 2026	Initial release of Engineering Coding Standards	Engineering	VP Engineering / CTO

53. Related Documents

- Coding Standards
- Engineering Handbook
- Git Workflow
- Code Review Guidelines
- Deployment Policy
- Testing Standards
- Information Security Policy